MACHINE LEARNING PROJECT

Loan Approval Prediction

Comparing Predictive Modeling Performance of Logistic Regression and Random Forest Classifiers

ABDELRAHMAN SHERIF
MAHMOUD ABBAS
MERNA AMR
NOUR BAHAA

Agenda

1 **Problem Statement**
What we're predicting, and why

2 **Dataset Overview**
loan_data.csv and its features

3 **Data Preprocessing**
Label encoding and data splits

4 **Model 1: Logistic Regression**
Linear baseline classifier

5 **Model 2: Random Forest**
Ensemble of decision trees

6 **Results & Comparison**
Accuracy side by side

7 **Conclusion & Next Steps**
Takeaways and future work

A Glimpse at the Data

45,000	14	22.2%
ROWS	COLUMNS	LOANS APPROVED

AGE	INCOME	LOAN AMT	INTENT	CREDIT SCORE	PRIOR DEFAULT	STATUS
22	$71,948	$35,000	Personal	561	No	Approved
21	$12,282	$1,000	Education	504	Yes	Denied
25	$12,438	$5,500	Medical	635	No	Approved
23	$79,753	$35,000	Medical	675	No	Approved
24	$66,135	$35,000	Medical	586	No	Approved
21	$12,951	$2,500	Venture	532	No	Approved

previous_loan_defaults_on_file, loan_intent, and other text fields are label-encoded to integers before training — shown here in original form for readability.

Problem Statement & Objective

Problem Statement

Lenders need to decide, quickly and consistently, whether an applicant is likely to repay a loan.

This project frames that decision as a binary classification problem: using an applicant's personal and financial profile to predict the target column `loan_status` (approved / not approved).

Two classifiers are trained on the same data and compared head-to-head on predictive accuracy.

PROJECT OBJECTIVE

- Clean & encode applicant data
- Train a Logistic Regression model
- Train a Random Forest model
- Compare accuracy on held-out test data
- Identify the stronger model for this task

Dataset Overview

`loan_data.csv` — one row per loan applicant, target column: `loan_status`

Numeric Applicant Features

- Age & years of employment
- Annual income
- Loan amount requested
- Loan interest rate
- Loan-to-income ratio
- Credit history length & credit score

Categorical Features (label-encoded)

- `person_gender`
- `person_education`
- `person_home_ownership`
- `loan_intent`
- `previous_loan_defaults_on_file`

Encoded to integers with scikit-learn's LabelEncoder before training.

Data Preprocessing

1. **Load Data**
   Read `loan_data.csv` into a pandas DataFrame

2. **Encode Categoricals**
   LabelEncoder applied to 5 categorical columns

3. **Standard Scaling**
   Scale numeric features for Logistic Regression convergence

4. **Train / Test Split**
   80% train / 20% test (random_state = 42)

PREPROCESSING CODE

```
# Train/Test Split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, ra

# StandardScaler implementation
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

# SMOTE Over-sampling setup
smote = SMOTE(random_state=42)
```

Model 1: Logistic Regression

A linear classifier baseline optimizing probability estimations. Scaling numerical features enables stable gradient descent step sizes, resulting in a significantly improved predictive baseline.

MODEL CODE

```
log = sk.linear_model.LogisticRegression(
    max_iter=2000, random_state=42
)
log.fit(x_train, y_train)
```

Standardized inputs • max_iter=2000 • trained on 80% of the data

TEST ACCURACY

89.1%

Critique: High performance baseline due to robust feature scaling, yet bounded by linear boundaries on non-linear categorical interactions.

Model 2: Random Forest

An ensemble of many decision trees, each trained on a random subset of data and features. Predictions are combined by majority vote, which captures non-linear relationships and reduces overfitting.

MODEL CODE

```
model = RandomForestClassifier(
    n_estimators=1400, random_state=52
)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
accuracy_score(y_test, y_pred)
```

1,400 trees in the ensemble • trained on 80% of the data

TEST ACCURACY

92.7%

Verdict: Best-performing model — the ensemble approach captures interactions the linear model misses.

Results & Comparison

KEY TAKEAWAY **+3.6 pts** accuracy gained by moving from a linear baseline classifier to an ensemble model.

Why Random Forest Wins

STRENGTHS OF THE WINNING MODEL

✓ Non-linear patterns

Trees split on feature thresholds, so the model captures interactions a straight decision boundary cannot.

✓ Ensemble averaging

1,400 trees vote together, which smooths out the noise and variance of any single tree.

✓ Robust to feature scale

Tree splits don't require scaled numeric inputs, unlike Logistic Regression's coefficients.

TRADE-OFFS TO KEEP IN MIND

✗ Less interpretable

Coefficients in Logistic Regression are easy to read; 1,400 trees are not.

✗ Heavier to train & serve

More trees means more memory and slower inference than one linear equation.

Conclusion & Next Steps

Random Forest outperformed Logistic Regression on this loan approval task, **92.7% vs 89.1%** test accuracy — confirming that this dataset has non-linear structure a linear model can't fully capture.

1 **SMOTE Optimization**
Optimize the SMOTE minority class ratio to check if balanced modeling boosts accuracy further.

2 **Cross-validation**
Use k-fold CV instead of one split for a more reliable accuracy estimate.

3 **Hyperparameter tuning**
Grid/random search over tree depth, count, and split criteria.

4 **Feature importance**
Inspect the Random Forest's feature importances to see what drives approval.

5 **Handle class imbalance**
Check the loan_status split and consider re-sampling or class weights.

6 **Try gradient boosting**
Compare against XGBoost / LightGBM for a stronger benchmark.